Image Processing and Computer Vision (CSC6051/MDS6004) Assignment 2

Wang Xuening
225040103
225040103@link.cuhk.edu.cn

Abstract

This report implements and evaluates three image processing and depth estimation tasks. Task 1 manually implements a Gaussian filter. Task 2 develops global and local histogram equalization, with a 31×31 window achieving balanced contrast enhancement. Task 3 performs monocular depth estimation on ScanNet. A baseline ResNet50 model achieves an AbsRel of 0.1247, while increasing training samples (160k) and reducing epochs (2) improves it to 0.0817. Two foundation models are compared: Depth Anything V2 (AbsRel 0.1219) performs comparably, while ZoeDepth (0.4565) shows domain mismatch. Results underscore the importance of data scale and pre-training alignment.

1. Task 1

Implementation Details The Gaussian filter[4] is implemented manually without high-level libraries. For kernel size k (odd, > 1) and standard deviation σ, the 2D kernel is:

$$G(i,j) = \exp\left(-\frac{(i-c)^2 + (j-c)^2}{2\sigma^2}\right), \quad c = \lfloor k/2 \rfloor.$$

If $\sigma = 0$, the kernel reduces to a unit impulse (center element = 1). The kernel is normalized to sum to 1. Zero padding is applied to handle borders, followed by pixel-wise convolution. For color images, the filter is applied independently to each channel. The output is clipped to $[0, 255]$ and converted to the original data type.

Results & Analysis Figure 1 shows the original image and the Gaussian filtering result ($k = 5, \sigma = 1$). The output is noticeably blurred, confirming effective suppression of high-frequency details. Larger kernel sizes or σ produce stronger smoothing. Zero padding introduces a subtle dark border, which is acceptable. The implementation behaves as expected.

Figure 1: Comparison before and after Gaussian filtering ($k = 5, \sigma = 1$)

We also note that different padding strategies (e.g., zero padding) can reduce boundary artifacts compared to zero padding, which may improve visual consistency near image borders.

Pseudocode The main steps of the Gaussian filter are summarized as follows:

```
function gaussian_filter(img, k,
    sigma):
    if sigma == 0:
        kernel = zero matrix of size
            k*k
        kernel[center, center] = 1
    else:
        for i in [0, k-1]:
            for j in [0, k-1]:
                kernel[i,j] = exp(-((
                    i-c)^2 + (j-c)^2)
                    /(2*sigma^2))
        kernel = kernel / sum(kernel)
    pad = floor(k/2)
    padded = pad image with zeros
    result = zero matrix of same size
         as img
    for i in [0, H-1]:
        for j in [0, W-1]:
            region = padded[i:i+k, j:
                j+k]
            result[i,j] = sum(region
                * kernel)
    return clip(result, 0, 255)
```

2. Task 2

Mathematical Formulation For ordinary histogram equalization[5], let H be the histogram of the input image with $L = 256$ bins. The cumulative distribution function (CDF) is $C(i) = \sum_{j=0}^{i} H(j)$. The mapping function that redistributes pixel intensities is:

$$T(i) = \left\lfloor \frac{C(i) - C_{\min}}{C_{\max} - C_{\min}} \times (L-1) \right\rfloor,$$

where $C_{\min}$ and $C_{\max}$ are the minimum and maximum non-zero CDF values. This mapping is applied via a lookup table.

For local histogram equalization, a sliding square window of odd size $w \times w$ is used. For each pixel, the histogram of its local neighborhood is computed, and the same transformation is applied using the CDF of that window. The window size 31×31 is chosen empirically to balance detail preservation and noise suppression: smaller windows amplify noise, while larger windows approach global equalization.

Results & Analysis Figure 2 presents the results of global and local histogram equalization. The top row shows (left to right) the original low-contrast image, the globally equalized result, and the locally equalized result (window size 31). The bottom row shows their corresponding histograms (horizontal axis: pixel intensity, vertical axis: number of pixels). Global equalization improves overall contrast but tends to over-enhance dark regions, where noise becomes more visible. In contrast, local equalization better preserves local details and achieves more balanced enhancement, especially under uneven illumination. The chosen window size of 31×31 offers a good trade-off between local adaptability and computational efficiency. Local equalization has higher complexity $O(N \times w^2)$ compared to global $O(N)$, reflecting its higher computational cost.

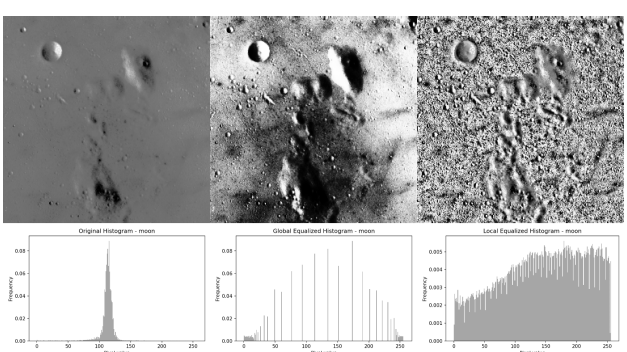


Figure 2: Comparison of histogram equalization methods.

Implementation Details Both methods are implemented without high-level libraries. For global equalization, we compute the histogram and cumulative distribution function (CDF) to build a lookup table. For local equalization, we apply zero padding and for each pixel compute the histogram of its 31×31 neighborhood, then map the pixel using the local CDF.

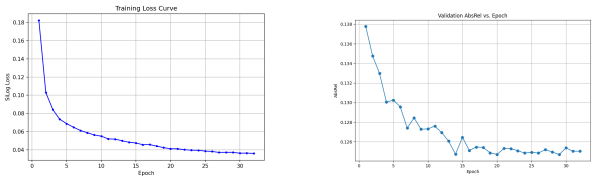
Pseudocode The main steps of the two histogram equalization methods are summarized as follows:

```
# Global histogram equalization
function histogram_equalization(img):
    if all pixels have the same value
        :
        return img
    hist = histogram of img (256 bins
        )
    cdf = cumulative sum of hist
    cdf_min = minimum non-zero CDF
        value
    cdf_max = maximum CDF value
    mapping = floor((cdf - cdf_min) *
         255 / (cdf_max - cdf_min))
    return mapping[img]

# Local histogram equalization
function local_histogram_equalization
    (img, window_size):
    pad = floor(window_size / 2)
    padded = zero-pad image with pad
    result = zeros_like(img)
    for each pixel (i, j) in img:
        window = padded[i:i+
            window_size, j:j+
            window_size]
        if all pixels in window are
            identical:
            result[i,j] = img[i,j]
        else:
            hist = histogram of
                window (256 bins)
            cdf = cumulative sum of
                hist
            cdf_min = minimum CDF
                value
            cdf_max = maximum CDF
                value
            mapping = floor((cdf -
                cdf_min) * 255 / (
                cdf_max - cdf_min))
            result[i,j] = mapping[img
                [i,j]]
    return result
```

3. Task 3

3.1. Baseline



(a) Training loss per epoch (b) Testing loss per epoch

Figure 3: Baseline training performanc

Training loss per epoch The model is optimized using the scale-invariant logarithmic (SiLog) loss[3], which is widely adopted for monocular depth estimation as it handles global scale ambiguity effectively. As shown in Fig. 3a, the average training loss steadily decreases from 0.1822 at epoch 1 to 0.0360 at epoch 32, demonstrating a stable convergence trend throughout the training process.

Test Abs_rel As illustrated in Fig. 3b, the absolute relative error (abs_rel) per epoch gradually decreases from 0.1378 at epoch 1 to 0.1250 at epoch 32, with minor fluctuations. The final average abs_rel is 0.1250, indicating that the model achieves reasonable accuracy on the test set, though some variability persists across epochs.

The decreasing training loss indicates stable optimization, while the gradual decline in test AbsRel suggests consistent generalization improvement, with minor fluctuations reflecting the presence of hard samples and domain variability.

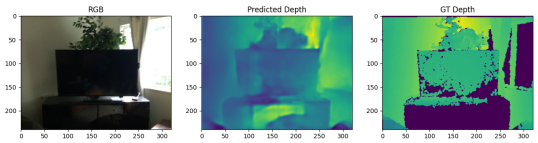
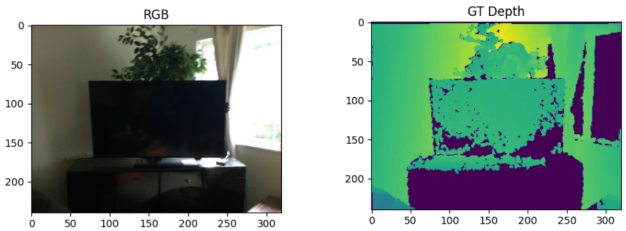


Figure 4: Comparison of methods

Figure 4 shows a qualitative comparison. From the depth prediction results, it can be observed that the model effectively captures the relative depth relationships of objects in the scene (e.g., the depth ordering of walls and tables). The predicted depth map is structurally consistent with the ground truth, though there is some smoothing at object boundaries and fine structures, which is a common phenomenon in monocular depth estimation.

3.2. Model Comparison

Figure 7 shows a randomly selected reference image: (a) the original RGB input; (b) the ground truth depth map. In the depth map, darker pixels represent closer objects, while brighter pixels represent farther objects, following a typical colormap. The two serve as the input and target for the depth estimation task, respectively.



(a) RGB input image (b) Ground truth depth

Figure 5: Reference images

Then I compare the baseline model (trained with default settings) against four configurations under different epoch–data combinations to investigate the data scale effect[6]. The quantitative results are summarized in Table 1.

Table 1: Quantitative comparison of different training configurations.

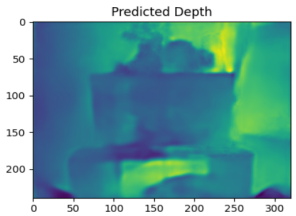
Configuration	Baseline	A	B	C	D
Training Samples	10,000	20,000	40,000	80,000	160,000
Total Epochs	32	16	8	4	2
Best Epoch	29	14	8	3	2
Test AbsRel	0.1247	0.1135	0.1023	0.0909	0.0817

From Table 1, increasing the number of training samples while reducing the number of epochs consistently improves the test AbsRel (Config A $\rightarrow$ D). The best performance is achieved with the largest dataset and the fewest epochs (Config D). In contrast, training longer on a smaller dataset does not guarantee better performance. For instance, the baseline (80k samples, 32 epochs) underperforms Config A (20k samples, 16 epochs), suggesting that excessive iterations may lead to overfitting or diminishing returns.

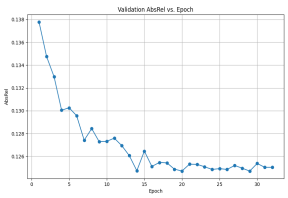
Overall, expanding the dataset yields more benefit than repeatedly iterating over a small dataset, though the improvement begins to saturate beyond 80,000 samples.

Figure 6 presents the predicted depth maps for a sample test image. Visual inspection shows that Config D (2 epochs, 160k samples) produces the sharpest object boundaries and the most accurate relative depth relationships, consistent with its lowest AbsRel (0.0817).

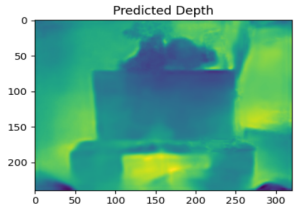
Analysis on Data Scale Effect The results reveal a clear trade-off between training epochs and dataset size. Given



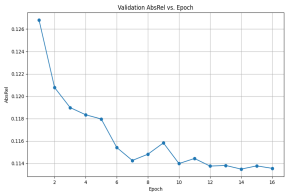
(a) Baseline
(32 epochs, 10k samples)



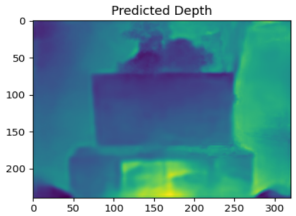
(b) Baseline depth
AbsRel: 0.1247



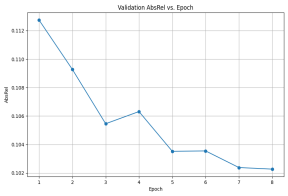
(c) Config A
(16 epochs, 20k samples)



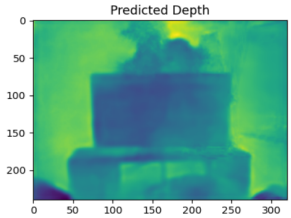
(d) Config A depth
AbsRel: 0.1135



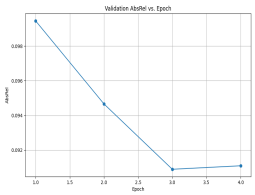
(e) Config B
(8 epochs, 40k samples)



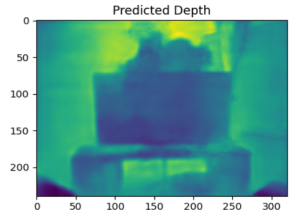
(f) Config B depth
AbsRel: 0.1023



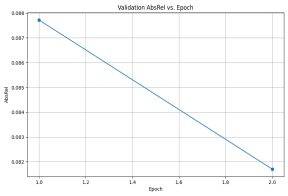
(g) Config C
(4 epochs, 80k samples)



(h) Config C depth
AbsRel: 0.0909



(i) Config D
(2 epochs, 160k samples)



(j) Config D depth
AbsRel: 0.0817

Figure 6: Comparison of training loss curves (left column) and predicted depth maps (right column) for five configurations. Each row corresponds to a different epoch–data combination, with the test AbsRel reported in parentheses.

a fixed computational budget, training on a larger dataset for fewer epochs is more beneficial than repeatedly iterating over a small dataset. While increasing data does not guarantee indefinite gains—performance tends to saturate once diversity is sufficient—within our experimental range, the largest dataset (160k samples) with the shortest training (2 epochs) achieves the best performance. This suggests that data scale plays a more critical role than training duration for this task, consistent with the general trend that increasing data diversity often improves generalization more effectively than additional iterations on limited data.

3.3. Foundation Model Comparison

To benchmark our baseline model against state-of-the-art 3D foundation models for depth estimation, we select two representative pre-trained models: **Depth Anything V2[7]** and **ZoeDepth**. Both models are trained on large-scale datasets and are known for their strong generalization capabilities.

Selected Foundation Models

- **Depth Anything V2[1]** : A vision foundation model pre-trained on diverse image datasets, adapted for monocular depth estimation via a lightweight decoder. We use the official pre-trained weights and follow the provided inference pipeline.

- **ZoeDepth** [1]: A depth-aware foundation model pre-trained on a mixture of indoor and outdoor datasets, specifically designed for metric depth prediction. We utilize the publicly available checkpoint and evaluation script.

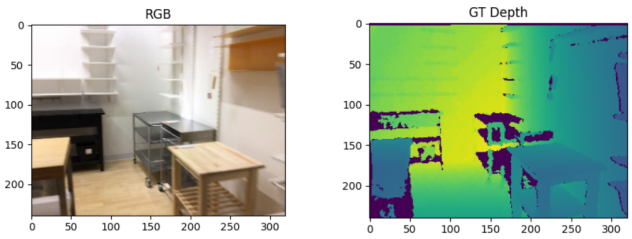
Evaluation Setup All models are evaluated on the ScanNet[2] test set using the same evaluation protocol as our baseline model. The input images are resized to 240×320 to match the baseline's input resolution. For models that output relative depth (e.g., Depth Anything V2), we apply a per-image scale-shift alignment before computing metrics. The primary metric is absolute relative error (AbsRel).

Quantitative Results Table 2 summarizes the performance. Our baseline model achieves an AbsRel of 0.1247, while Depth Anything V2 yields 0.1219, slightly better than the baseline. In contrast, ZoeDepth produces a significantly higher error of 0.4565. This indicates that while Depth Anything V2 generalizes well to the ScanNet domain, ZoeDepth struggles to produce accurate metric depth in this indoor scene, possibly due to domain mismatch.

Table 2: Quantitative comparison on the ScanNet test set.

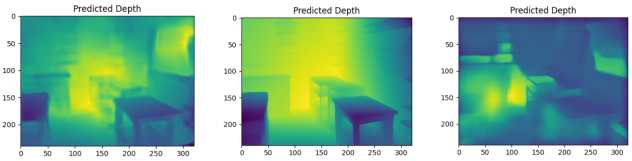
Model	AbsRel
Baseline	0.1247
Depth Anything V2	0.1219
ZoeDepth	0.4565

Qualitative Comparison Before presenting the predicted depth maps, we first show the input RGB image and its corresponding ground truth depth map in Figure 7. This provides a visual baseline for comparing the predictions.



(a) RGB input (b) Ground truth depth

Figure 7: Reference images: (a) original RGB image; (b) corresponding ground truth depth map.



(a) Baseline (b) Depth Anything V2 (c) ZoeDepth

Figure 8: Predicted depth maps for the same ScanNet image.

The performance gap between models can be largely attributed to the difference between relative and metric depth prediction, as well as domain alignment. Depth Anything V2, trained on large-scale diverse data, provides strong generalizable features and achieves performance comparable to the baseline after scale-shift alignment. In contrast, ZoeDepth directly predicts metric depth but shows poor generalization to ScanNet, likely due to domain mismatch between its pre-training data and indoor scenes.

These results suggest that domain alignment plays a critical role in depth estimation, and that large-scale pre-training can partially compensate for limited task-specific supervision. However, foundation models typically require higher computational resources, which may limit their practical deployment.

4. Conclusion

In this assignment, three tasks in image processing and monocular depth estimation were implemented and evaluated.

A Gaussian filter was constructed from scratch, demonstrating correct kernel-based smoothing behavior. Global and local histogram equalization methods were developed, with the local variant (31×31) achieved more balanced contrast enhancement.

For monocular depth estimation on ScanNet, the baseline ResNet50 model achieved an AbsRel of 0.1247. Experiments on data scale showed that increasing training data while reducing epochs consistently improved performance, reaching 0.0817. Comparisons with foundation models revealed that Depth Anything V2 performs competitively with the baseline, while ZoeDepth suffers from domain mismatch.

Overall, the results highlight the importance of data scale and domain alignment in depth estimation.

Code Usage Instructions

All code is implemented in Python and tested with PyTorch 2.0+ and CUDA 12.4. The project is organized as follows:

```
.
task1/
    task1.py          # Gaussian filter
task2/
    task2.py          # Histogram equalization
task3/                # Monocular depth estimation
    train.py # Baseline training(config A~D)
    test.py           # Baseline evaluation
    depth_model.py    # ResNet50 model definition
    scannet_dataset.py # ScanNet data loader
    metrics.py        # Evaluation metrics &
     alignment
    eval_foundation.py # evaluation for
     foundation models(t3.4)
    t3.4.sh # run eval_foundation.py for 2 models
    run.sh            # config A~D
    test_baseline.sh  # basline test
    baseline/         # Stored baseline data

t3.3/  # Data scale experiment results
    20000_16_opt/  # Config A (20k samples, 16
     epochs)
    40000_8_opt/   # Config B
    80000_4_opt/   # Config C
    160000_2_opt/  # Config D
    test_*.sh      # Evaluation scripts for each
     config
t3.4/              # Foundation models
    d_any/Depth-Anything-V2/ # Depth Anything V2
     (conda env: d_zoe)
        checkpoints/# Pre-trained weight
        depth_anything_wrapper.py
    d_zoe/ZoeDepth/# ZoeDepth (conda env: d_zoe)
        checkpoints/# Pre-trained weight
        zoe_wrapper.py
```

How to Run

- **Task 1 & 2**: Execute `task1.py` and `task2.py` directly.
- **Task 3 – Baseline Training**: Use `run.sh` to train the baseline model (including config A D). Modify the script to adjust training samples and epochs for the data scale study.

- **Task 3 – Baseline Evaluation**: Use `test_baseline.sh` to evaluate a trained checkpoint.
- **Task 3 – Data Scale Study**: The training configurations are stored in `t3.3/`. Use the provided `test_*.sh` scripts to evaluate each configuration.
- **Task 3 – Foundation Model Comparison**: Activate the corresponding conda environment (`d_zoe`), then run `eval_foundation.py` with appropriate arguments (see comments in the script or the example `.sh` files in task3/t3.4.sh).

All results (metrics and visualizations) are saved in the respective output directories.

References

[1] Shariq Farooq Bhat, Reiner Birkl, Diana Wofk, Peter Wonka, and Matthias Müller. Zoedepth: Zero-shot transfer by combining relative and metric depth. *arXiv preprint arXiv:2302.12288*, 2023. 4

[2] Angela Dai, Angel X Chang, Manolis Savva, Maciej Halber, Thomas Funkhouser, and Matthias Nießner. Scannet: Richly-annotated 3d reconstructions of indoor scenes. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 5828–5839, 2017. 4

[3] David Eigen, Christian Puhrsch, and Rob Fergus. Depth map prediction from a single image using a multi-scale deep network. *Advances in neural information processing systems*, 27, 2014. 3

[4] Rafael C Gonzalez. *Digital image processing*. Pearson education india, 2009. 1

[5] Robert Hummel. Image enhancement by histogram transformation. *Unknown*, 1975. 2

[6] Chen Sun, Abhinav Shrivastava, Saurabh Singh, and Abhinav Gupta. Revisiting unreasonable effectiveness of data in deep learning era. In *Proceedings of the IEEE international conference on computer vision*, pages 843–852, 2017. 3

[7] Lihe Yang, Bingyi Kang, Zilong Huang, Zhen Zhao, Xiaogang Xu, Jiashi Feng, and Hengshuang Zhao. Depth anything v2. *Advances in Neural Information Processing Systems*, 37:21875–21911, 2024. 4